离散数学：高级计数技术 (Advanced Counting Techniques) 复习讲义

整理自：第 8 章 PPT 与 DS-qian-8-2026 讲义

目录

1 本章导引 **3**
1.1 学习目标 . . . 3

2 递推关系、线性递推与分治 **3**
2.1 递推关系的定义 . . . 3
2.2 基本例子 . . . 4
2.3 递推关系与递归定义 . . . 5
2.4 用递推关系建模 . . . 5
2.5 动态规划 . . . 7
2.6 线性递推关系求解 . . . 10
2.7 求解方法概览 . . . 10
2.8 线性齐次常系数递推关系 . . . 10
2.9 判断例子 . . . 10
2.10 特征方程 . . . 11
2.11 一阶递推 . . . 11
2.12 二阶递推：不同根 . . . 11
2.13 二阶递推：重根 . . . 12
2.14 Fibonacci 数的显式公式 . . . 12
2.15 重根例子 . . . 12
2.16 k 阶递推的一般结论 . . . 13
2.17 线性非齐次常系数递推关系 . . . 13
2.18 非齐次递推的结构 . . . 14
2.19 待定系数法 . . . 14
2.20 非齐次例子：前 n 个正整数之和 . . . 14
2.21 非齐次例子：$a_n = 3a_{n-1} + 2n$ . . . 15
2.22 非齐次例子：指数项 . . . 16

2.23 分治算法与递推关系 16
2.24 分治思想 16
2.25 分治递推 16
2.26 分治递推定理一 18
2.27 定理一的展开证明思路 18
2.28 例子 18
2.29 Master Theorem 19
2.30 归并排序算法与复杂度 19
2.31 最近点对问题 19

3 生成函数 **20**
3.1 定义 20
3.2 基本例子 20
3.3 形式幂级数的运算 20
3.4 常用展开 21
3.5 广义二项式系数 21
3.6 组合计数中的生成函数 22
3.7 整数解例子 22
3.8 自动售货机硬币例子 23
3.9 指数生成函数与排列 24
3.10 部分分式分解与系数读取 24
3.11 用生成函数解递推 26
3.12 用生成函数证明恒等式 27

4 容斥原理及其应用 **28**
4.1 基本形式 28
4.2 三集合形式 28
4.3 语言课程例子 28
4.4 容斥原理的应用 29
4.5 性质形式 29
4.6 有上界的非负整数解 29
4.7 Eratosthenes 筛法 29
4.8 满射数量 30
4.9 错排 30
4.10 帽子寄存问题 30

5 本章总结 (Summary) **31**
5.1 PPT 作业汇总 31
5.2 术语对照表 32

1　本章导引

本章讨论在基础计数技术之外更系统、更有表达力的计数方法。PPT 开头给出的三个问题分别对应本章三条主线：

- 长度为 n 的二进制串中，不含两个连续 0 的字符串有多少个？这类问题通常用**递推关系 (recurrence relation)** 建模。
- 怎样用形式幂级数的系数表示一列数？这引出**生成函数 (generating function)**。
- 将 7 个工作分配给 3 名员工，并要求每名员工至少分到一个工作，有多少种分配方式？这类“至少、至多、不能同时发生”的限制常用**容斥原理 (inclusion-exclusion)**。

本章结构

第 8 章包括：递推关系的应用、线性递推关系求解、分治算法中的递推、生成函数、容斥原理以及容斥的应用。它们和第 6 章的排列、组合、二项式定理等基础计数方法结合，可以处理大量复杂计数问题。

1.1　学习目标

学完本章后应能做到：

1. 根据计数问题建立递推关系，并写出必要的初始条件；
2. 判断递推关系是否线性、齐次、常系数，并用特征方程求解；
3. 用递推关系分析二分查找、归并排序、快速整数乘法等分治算法的复杂度；
4. 熟练使用普通生成函数和指数生成函数解决组合与排列计数问题；
5. 使用容斥原理处理“至少一个条件不满足”“不能固定点”“满射”等问题。

2　递推关系、线性递推与分治

2.1　递推关系的定义

递推关系 (recurrence relation) 是用序列前面若干项表示后一项的方程。设序列为 $\{a_n\}$，若存在非负整数 n_0，使得当 $n \geq n_0$ 时

$$a_n = f(a_0, a_1, \ldots, a_{n-1}),$$

则称该方程是序列 $\{a_n\}$ 的递推关系。

- 若某个序列的每一项都满足该递推关系，则称该序列是递推关系的一个**解 (solution)**。
- **初始条件 (initial conditions)** 指递推关系开始生效前必须给出的若干项。

- 递推关系和足够的初始条件通常可以唯一确定一个序列。

核心理解

递推关系本质上是“当前项由之前项决定”。递推关系给出规则，初始条件给出起点；两者合在一起才能唯一确定序列。

2.2 基本例子

2.2.1 Fibonacci 序列

Fibonacci 序列满足：

$$f_n = f_{n-1} + f_{n-2}.$$

如果指定 $f_0 = 0$, $f_1 = 1$，则序列唯一确定为

$$0, 1, 1, 2, 3, 5, 8, \ldots$$

这是一个二阶递推关系，因为 f_n 依赖前两项。

2.2.2 Pascal 恒等式

二项式系数满足二维递推：

$$\binom{n+1}{k} = \binom{n}{k} + \binom{n}{k-1}.$$

它说明 Pascal 三角形中的每个内部元素都等于其上一行相邻两个元素之和。要唯一确定整个三角形，需要给出边界条件，例如

$$\binom{n}{0} = 1, \qquad \binom{n}{n} = 1.$$

2.2.3 检验一个序列是否为解

考虑递推关系

$$a_n = 2a_{n-1} - a_{n-2}, \qquad n \geq 2.$$

若 $a_n = 3n$，则

$$2a_{n-1} - a_{n-2} = 2 \cdot 3(n-1) - 3(n-2) = 3n = a_n,$$

所以它是一个解。若 $a_n = 2^n$，则

$$2a_{n-1} - a_{n-2} = 2 \cdot 2^{n-1} - 2^{n-2} = 3 \cdot 2^{n-2} \neq 2^n,$$

所以不是解。若 $a_n = 5$，两边都等于 5，所以它也是一个解。

2.3 递推关系与递归定义

递推关系是递归定义的一种特殊形式。递归定义可以定义对象、集合或算法；当它用于描述序列中后一项如何由前面项得到时，就称为递推关系。

初始条件个数

若一个递推关系的阶数为 k，通常需要 k 个独立初始条件。例如 Fibonacci 递推需要 f_0 和 f_1；若

$$a_n = a_{n-1} + a_{n-8},$$

则为了从低到高推出所有项，需要给出 $a_0, a_1, \ldots, a_7$。

2.4 用递推关系建模

2.4.1 细菌增长

若一个细菌群每小时数量翻倍，初始有 5 个细菌，设 B_n 为 n 小时后的数量，则

$$B_n = 2B_{n-1}, \qquad B_0 = 5.$$

由此可得显式公式：

$$B_n = 5 \cdot 2^n.$$

2.4.2 复利问题

若向银行存入 $\$10,000$，年利率为 11%，每年复利一次，设 P_n 为 n 年后的金额，则

$$P_n = P_{n-1} + 0.11P_{n-1} = 1.11P_{n-1}, \qquad P_0 = 10000.$$

因此

$$P_n = 10000(1.11)^n.$$

特别地，

$$P_{30} = 10000(1.11)^{30}.$$

2.4.3 兔子繁殖

一对小兔出生后两个月才开始繁殖，之后每个月产生一对新兔，且兔子不死亡。设 f_n 为第 n 个月后的兔子对数，则

$$f_n = f_{n-1} + f_{n-2}, \qquad n > 2,$$

初始条件为

$$f_1 = 1, \qquad f_2 = 1.$$

这正是 Fibonacci 型递推。

2.4.4 汉诺塔

设 H_n 为移动 n 个盘子所需的最少步数。为了移动最大盘，必须先把上面的 $n-1$ 个盘移到辅助柱，再移动最大盘，最后把 $n-1$ 个盘移到目标柱。因此：

$$H_n = 2H_{n-1} + 1, \qquad H_1 = 1.$$

迭代展开：

$$\begin{aligned} H_n &= 2H_{n-1} + 1 \\ &= 2(2H_{n-2} + 1) + 1 \\ &= 2^2 H_{n-2} + 2 + 1 \\ &= \cdots \\ &= 2^{n-1} H_1 + 2^{n-2} + \cdots + 2 + 1 \\ &= 2^n - 1. \end{aligned}$$

2.4.5 不含连续 0 的二进制串

设 a_n 为长度为 n 且不含两个连续 0 的二进制串数量。按最后一位分类：

- 若最后一位是 1，前 $n-1$ 位可以是任意合法串，共 a_{n-1} 个；
- 若最后一位是 0，倒数第二位必须是 1，前 $n-2$ 位可以是任意合法串，共 a_{n-2} 个。

所以

$$a_n = a_{n-1} + a_{n-2}, \qquad a_1 = 2, \quad a_2 = 3.$$

2.4.6 偶数个 0 的十进制码字

一个十进制数字串若含有偶数个 0，称为有效码字。设 a_n 为长度为 n 的有效码字数量。考虑在长度 $n-1$ 的串后追加一位：

- 若前缀已有偶数个 0，最后一位可取 1 到 9，有 $9a_{n-1}$ 种；
- 若前缀有奇数个 0，最后一位必须取 0。长度 $n-1$ 的所有串共有 10^{n-1} 个，所以奇数个 0 的串有 $10^{n-1} - a_{n-1}$ 个。

因此

$$a_n = 9a_{n-1} + (10^{n-1} - a_{n-1}) = 8a_{n-1} + 10^{n-1}.$$

2.4.7 括号化与 Catalan 数

设 C_n 为将 $n+1$ 个数

$$x_0 \cdot x_1 \cdot x_2 \cdots x_n$$

完全加括号以指定乘法顺序的方式数。最后一次乘法必然把表达式切分为左右两段：

$$(x_0 \cdots x_k) \cdot (x_{k+1} \cdots x_n).$$

左侧有 C_k 种括号化方式，右侧有 C_{n-k-1} 种括号化方式，因此

$$C_n = \sum_{k=0}^{n-1} C_k C_{n-k-1}, \qquad C_0 = 1.$$

这就是 Catalan 数的典型递推。PPT 中给出 $C_3 = 5$，对应四个数的五种完全括号化方式。

2.5 动态规划

动态规划 (dynamic programming, DP) 是一种通常建立在递推关系和初始状态之上的算法技术。它把问题拆成子问题，保存已经求出的子问题答案，再由这些答案构造更大问题的答案。

DP 的常见要素

写动态规划时通常按四步整理：状态定义、状态转移、初始化、答案位置。若递归版本会重复计算大量子问题，则通过表格保存结果的过程称为**记忆化 (memoization)**。

2.5.1 一维 DP：最长递增子序列

给定整数数组 `nums`，求最长严格递增子序列的长度。例如

$$[10, 9, 2, 5, 3, 7, 101, 18]$$

的最长递增子序列可以是 $[2, 3, 7, 101]$ 或 $[2, 5, 7, 18]$，长度为 4。

定义状态：

$$dp[i] = \text{以} nums[i] \text{ 结尾的最长递增子序列长度}.$$

对每个 i，检查所有 $j < i$。若 $nums[j] < nums[i]$，则可将 $nums[i]$ 接在以 $nums[j]$ 结尾的递增子序列之后：

$$dp[i] = \max(dp[i], dp[j] + 1).$$

初始化为 $dp[i] = 1$，最终答案为

$$\max_{0 \le i < n} dp[i].$$

Listing 1: 最长递增子序列的 $O(n^2)$ 动态规划

```
def lengthOfLIS(nums):
    if not nums:
        return 0
    n = len(nums)
    dp = [1] * n
```

```
    for i in range(n):
        for j in range(i):
            if nums[j] < nums[i]:
                dp[i] = max(dp[i], dp[j] + 1)
    return max(dp)

nums = [10, 9, 2, 5, 3, 7, 101, 18]
print(lengthOfLIS(nums))  # 4
```

补充：$O(n \log n)$ 优化

LIS 还可以用“维护各长度递增子序列的最小结尾值”加二分查找优化到 $O(n \log n)$。这个方法不直接保存每个位置的最优值，而是保存更适合扩展的候选结尾。

2.5.2 二维 DP：最长公共子序列

给定两个字符串 `text1` 和 `text2`，求它们最长公共子序列的长度。子序列不要求连续，但字符相对顺序必须一致。例如：

`text1="abcde", text2="ace"`

最长公共子序列为 `"ace"`，长度为 3。

定义：

$$dp[i][j] = \texttt{text1}\ \text{前}i\ \text{个字符与}\ \texttt{text2}\ \text{前}j\ \text{个字符的 LCS 长度.}$$

状态转移：

$$dp[i][j] = \begin{cases} dp[i-1][j-1] + 1, & \text{若} text1[i-1] = text2[j-1], \\ \max(dp[i-1][j], dp[i][j-1]), & \text{否则.} \end{cases}$$

Listing 2: 最长公共子序列的二维动态规划

```
def longestCommonSubsequence(text1: str, text2: str) -> int:
    m, n = len(text1), len(text2)
    dp = [[0] * (n + 1) for _ in range(m + 1)]

    for i in range(1, m + 1):
        for j in range(1, n + 1):
            if text1[i - 1] == text2[j - 1]:
                dp[i][j] = dp[i - 1][j - 1] + 1
            else:
                dp[i][j] = max(dp[i - 1][j], dp[i][j - 1])
    return dp[m][n]
```

该算法时间复杂度为 $O(mn)$，空间复杂度为 $O(mn)$。若只需要长度，可以将空间优化到 $O(\min(m, n))$。

2.5.3 带权区间调度：讲座安排

PPT 中的 talk schedule 是动态规划的典型例子。设有 n 个讲座，讲座 j 的开始时间为 s_j，结束时间为 e_j，听众数为 w_j。目标是在不重叠的前提下选出若干讲座，使总听众数最大。

先按结束时间排序：

$$e_1 \le e_2 \le \cdots \le e_n.$$

若两个讲座时间不重叠，则称它们**兼容 (compatible)**。定义 $p(j)$ 为小于 j 且与讲座 j 兼容的最大下标：

$$p(j) = \max\{i \mid i < j,\ e_i \le s_j\}.$$

若不存在这样的 i，则令 $p(j) = 0$。

设

$$T(j) = \text{只考虑前} j \text{ 个讲座时可获得的最大总听众数.}$$

最优方案有两种可能：

- 选讲座 j：则还可以选的只能来自前 $p(j)$ 个讲座，总数为 $w_j + T(p(j))$；
- 不选讲座 j：答案与前 $j-1$ 个讲座相同，为 $T(j-1)$。

所以递推关系为

$$T(j) = \max\{w_j + T(p(j)),\, T(j-1)\}, \qquad T(0) = 0.$$

Listing 3: 讲座安排的动态规划框架

```
MaximumAttendees(s1, s2, ..., sn, e1, e2, ..., en, w1, w2, ..., wn):
    sort talks by end time and relabel so that e1 <= e2 <= ... <= en

    for j = 1 to n:
        if no talk i with i < j is compatible with talk j:
            p[j] = 0
        else:
            p[j] = max { i | i < j and talk i is compatible with talk j }

    T[0] = 0
    for j = 1 to n:
        T[j] = max(w[j] + T[p[j]], T[j - 1])

    return T[n]
```

PPT 中 $p(j)$ 的例子

示意图中有 7 个讲座，按结束时间排序后给出：

$$p(1)=0,\ p(2)=0,\ p(3)=1,\ p(4)=0,\ p(5)=0,\ p(6)=2,\ p(7)=4.$$

这些值表示每个讲座之前“最后一个能与它兼容”的讲座编号。

2.6 线性递推关系求解

2.7 求解方法概览

递推关系通常并不容易直接求闭式公式。本节讨论一类特别重要、可以系统求解的递推关系：

- 线性齐次常系数递推关系；
- 线性非齐次常系数递推关系。

常见求解方法包括迭代展开、生成函数，以及本节重点使用的**特征方程法**。

2.8 线性齐次常系数递推关系

线性齐次常系数递推关系 (linear homogeneous recurrence relation with constant coefficients, LHRRCC) 的一般形式为：

$$a_n = c_1 a_{n-1} + c_2 a_{n-2} + \cdots + c_k a_{n-k},$$

其中 $c_1, c_2, \ldots, c_k$ 为实数，且 $c_k \neq 0$。

术语	含义
线性	右侧是前面若干项的线性组合，不含 a_i^2、$a_i a_j$ 等非线性项
常系数	系数 c_i 不依赖 n
阶数 k	a_n 最远依赖到 a_{n-k}
齐次	将所有 a_i 项移到左侧后，右侧为 0

2.9 判断例子

- $a_n = (1.02)a_{n-1}$：线性、常系数、齐次、一阶。
- $a_n = (1.02)a_{n-1} + 2^{n-1}$：线性、常系数、非齐次、一阶。
- $a_n = a_{n-1} + a_{n-2} + a_{n-3} + 2$：线性、常系数、非齐次、三阶。
- $a_n = n^2 a_{n-1} + n a_{n-2} + a_{n-1}a_{n-2}$：非线性，且系数不是常数。

- $H_n = 2H_{n-1} + 1$：线性常系数，但非齐次。
- $B_n = nB_{n-1}$：线性齐次，但不是常系数。

易错点

“齐次”只看有没有额外的 $F(n)$ 项；“常系数”只看乘在 a_{n-i} 前面的系数是否依赖 n；“线性”看是否出现前项之间的乘积或幂。

2.10 特征方程

对递推关系

$$a_n = c_1 a_{n-1} + c_2 a_{n-2} + \cdots + c_k a_{n-k},$$

假设存在形如

$$a_n = r^n$$

的解。代入得：

$$r^n = c_1 r^{n-1} + c_2 r^{n-2} + \cdots + c_k r^{n-k}.$$

两边除以 r^{n-k}，得到特征方程：

$$r^k - c_1 r^{k-1} - c_2 r^{k-2} - \cdots - c_k = 0.$$

该方程的根称为递推关系的**特征根 (characteristic roots)**。

2.11 一阶递推

对于

$$a_n = c a_{n-1},$$

特征方程为 $r - c = 0$，所以通解为

$$a_n = \alpha c^n.$$

再用初始条件确定 α。例如若 $a_n = 3a_{n-1}$ 且 $a_0 = 4$，则

$$a_n = 4 \cdot 3^n.$$

2.12 二阶递推：不同根

对于

$$a_n = c_1 a_{n-1} + c_2 a_{n-2},$$

特征方程为

$$r^2 - c_1 r - c_2 = 0.$$

若它有两个不同根 r_1, r_2，则通解为

$$a_n = \alpha_1 r_1^n + \alpha_2 r_2^n.$$

初始条件 a_0, a_1 给出：

$$\begin{cases} a_0 = \alpha_1 + \alpha_2, \\ a_1 = \alpha_1 r_1 + \alpha_2 r_2. \end{cases}$$

解出 α_1, α_2 后即可得到显式公式。

2.13 二阶递推：重根

若特征方程只有一个根 r_0，且重数为 2，则通解不是简单的 αr_0^n，而是

$$a_n = \alpha_1 r_0^n + \alpha_2 n r_0^n = (\alpha_1 + \alpha_2 n) r_0^n.$$

为什么重根要乘 n

如果特征方程有重根，只有 r_0^n 一个形状不足以满足二阶递推的两个初始条件。乘上 n 得到第二个线性无关解 nr_0^n。

2.14 Fibonacci 数的显式公式

Fibonacci 递推为

$$f_n = f_{n-1} + f_{n-2}, \qquad f_0 = 0, \quad f_1 = 1.$$

特征方程为

$$r^2 - r - 1 = 0,$$

两个根为

$$r_1 = \frac{1+\sqrt{5}}{2}, \qquad r_2 = \frac{1-\sqrt{5}}{2}.$$

所以

$$f_n = \alpha_1 \left(\frac{1+\sqrt{5}}{2}\right)^n + \alpha_2 \left(\frac{1-\sqrt{5}}{2}\right)^n.$$

由初始条件解得

$$\alpha_1 = \frac{1}{\sqrt{5}}, \qquad \alpha_2 = -\frac{1}{\sqrt{5}}.$$

因此

$$f_n = \frac{1}{\sqrt{5}} \left(\frac{1+\sqrt{5}}{2}\right)^n - \frac{1}{\sqrt{5}} \left(\frac{1-\sqrt{5}}{2}\right)^n.$$

2.15 重根例子

求解

$$a_n = 6a_{n-1} - 9a_{n-2}, \qquad a_0 = a_1 = 1.$$

特征方程为

$$r^2 - 6r + 9 = (r-3)^2 = 0.$$

通解为

$$a_n = (\alpha_1 + \alpha_2 n)3^n.$$

由 $a_0 = 1$ 得 $\alpha_1 = 1$；由 $a_1 = 1$ 得

$$3(1 + \alpha_2) = 1, \qquad \alpha_2 = -\frac{2}{3}.$$

所以

$$a_n = \left(1 - \frac{2n}{3}\right) 3^n.$$

2.16　k 阶递推的一般结论

若特征方程

$$r^k - c_1 r^{k-1} - \cdots - c_k = 0$$

有 k 个互不相同的根 $r_1, r_2, \ldots, r_k$，则通解为

$$a_n = \alpha_1 r_1^n + \alpha_2 r_2^n + \cdots + \alpha_k r_k^n.$$

若特征方程有 t 个不同根 $r_1, \ldots, r_t$，其重数分别为 $m_1, \ldots, m_t$，且

$$m_1 + m_2 + \cdots + m_t = k,$$

则通解为

$$a_n = \sum_{i=1}^{t} \left(\alpha_{i,0} + \alpha_{i,1} n + \cdots + \alpha_{i,m_i-1} n^{m_i-1}\right) r_i^n.$$

记忆方法

每个根 r 贡献一组 $r^n, nr^n, n^2 r^n, \ldots$。根的重数是几，就从 $n^0 r^n$ 一直写到 $n^{m-1} r^n$。

2.17　线性非齐次常系数递推关系

线性非齐次常系数递推关系的一般形式为：

$$a_n = c_1 a_{n-1} + c_2 a_{n-2} + \cdots + c_k a_{n-k} + F(n),$$

其中 $F(n)$ 不是恒等于 0 的函数。对应的**关联齐次递推关系**为：

$$a_n = c_1 a_{n-1} + c_2 a_{n-2} + \cdots + c_k a_{n-k}.$$

常见例子包括：

$$a_n = a_{n-1} + 2^n, \qquad a_n = a_{n-1} + a_{n-2} + n^2 + n + 1, \qquad a_n = 3a_{n-1} + n3^n.$$

2.18　非齐次递推的结构

若 $\{a_n^{(p)}\}$ 是非齐次递推的一个特解，$\{a_n^{(h)}\}$ 是关联齐次递推的通解，则非齐次递推的全部解为

$$a_n = a_n^{(p)} + a_n^{(h)}.$$

证明思路

若 b_n 和 $a_n^{(p)}$ 都是非齐次递推的解，将它们对应方程相减，非齐次项 $F(n)$ 会抵消，因此 $b_n - a_n^{(p)}$ 满足关联齐次递推。

2.19　待定系数法

若

$$F(n) = (b_t n^t + b_{t-1} n^{t-1} + \cdots + b_1 n + b_0) s^n,$$

则特解通常尝试：

$$(p_t n^t + p_{t-1} n^{t-1} + \cdots + p_1 n + p_0) s^n.$$

若 s 是关联齐次特征方程的 m 重根，则需要再乘 n^m：

$$n^m (p_t n^t + p_{t-1} n^{t-1} + \cdots + p_1 n + p_0) s^n.$$

2.19.1　特解形状例子

考虑

$$a_n = 6a_{n-1} - 9a_{n-2} + F(n).$$

关联齐次方程的特征根为 3，重数为 2，所以齐次通解为

$$a_n^{(h)} = (\alpha_1 + \alpha_2 n) 3^n.$$

不同 $F(n)$ 对应的特解形状如下：

$F(n)$	可尝试的特解形状
3^n	$p_0 n^2 3^n$
$n3^n$	$n^2(p_1 n + p_0)3^n$
$n^2 2^n$	$(p_2 n^2 + p_1 n + p_0)2^n$
$(n^2+1)3^n$	$n^2(p_2 n^2 + p_1 n + p_0)3^n$

2.20　非齐次例子：前 n 个正整数之和

设 a_n 是前 n 个正整数之和，则

$$a_n = a_{n-1} + n, \qquad a_1 = 1.$$

关联齐次递推为 $a_n = a_{n-1}$，通解为 $a_n^{(h)} = c$。由于 $F(n) = n$ 是一次多项式，且 $s = 1$ 是特征根，需尝试二次多项式中无常数项的形式：

$$a_n^{(p)} = p_1 n^2 + p_0 n.$$

代入

$$p_1 n^2 + p_0 n = p_1 (n-1)^2 + p_0 (n-1) + n$$

得到

$$p_1 = p_0 = \frac{1}{2}.$$

因此

$$a_n = c + \frac{1}{2} n^2 + \frac{1}{2} n.$$

由 $a_1 = 1$ 得 $c = 0$，所以

$$a_n = \frac{n(n+1)}{2}.$$

2.21 非齐次例子：$a_n = 3a_{n-1} + 2n$

求

$$a_n = 3a_{n-1} + 2n, \qquad a_1 = 3.$$

关联齐次解为

$$a_n^{(h)} = \alpha 3^n.$$

尝试特解 $p_n = cn + d$，代入：

$$cn + d = 3(c(n-1) + d) + 2n.$$

比较系数得

$$c = -1, \qquad d = -\frac{3}{2}.$$

故

$$a_n = -n - \frac{3}{2} + \alpha 3^n.$$

由 $a_1 = 3$ 得

$$-1 - \frac{3}{2} + 3\alpha = 3, \qquad \alpha = \frac{11}{6}.$$

所以

$$a_n = -n - \frac{3}{2} + \frac{11}{6} 3^n.$$

2.22 非齐次例子：指数项

求

$$a_n = 5a_{n-1} - 6a_{n-2} + 7^n.$$

关联齐次特征方程为

$$r^2 - 5r + 6 = (r-2)(r-3) = 0.$$

齐次通解为

$$a_n^{(h)} = \alpha_1 3^n + \alpha_2 2^n.$$

因为 $F(n) = 7^n$，且 7 不是特征根，尝试特解 $C7^n$。代入得

$$C7^n = 5C7^{n-1} - 6C7^{n-2} + 7^n.$$

两边除以 7^{n-2}：

$$49C = 35C - 6C + 49,$$

所以

$$C = \frac{49}{20}.$$

全部解为

$$a_n = \alpha_1 3^n + \alpha_2 2^n + \frac{49}{20}7^n.$$

2.23 分治算法与递推关系

2.24 分治思想

许多递归算法会把规模为 n 的问题拆成一个或多个较小的同类问题。分治算法 (divide-and-conquer) 的典型过程为：

1. Divide：把原问题拆成若干规模较小的同类子问题；
2. Conquer：递归求解这些子问题；
3. Combine：把子问题的解合成为原问题的解。

分治与动态规划

分治通常强调把问题拆成相对独立的子问题；动态规划则特别强调子问题重叠，并保存子问题答案以避免重复计算。

2.25 分治递推

常见分治递推形式为：

$$f(n) = af(n/b) + g(n).$$

其中：

- a 是子问题个数;
- 每个子问题规模约为 n/b;
- $g(n)$ 是划分与合并所需的额外代价。

2.25.1 二分查找

二分查找每次将问题规模减半。PPT 中用

$$f(n) = f(n/2) + 2$$

表示偶数规模时的比较次数，其中两个比较分别用于判断进入哪一半以及是否还有元素需要检查。由后面的定理可知:

$$f(n) = O(\log n).$$

2.25.2 归并排序

归并排序把数组拆成两半，分别排序后再线性合并:

$$M(n) = 2M(n/2) + n.$$

由 Master Theorem 可得:

$$M(n) = O(n \log n).$$

2.25.3 快速整数乘法

设 a, b 是两个 $2n$ 位二进制整数，拆成高低两半:

$$a = 2^n A_1 + A_0, \qquad b = 2^n B_1 + B_0.$$

普通做法需要计算 $A_1B_1, A_1B_0, A_0B_1, A_0B_0$ 四个 n 位乘法。PPT 给出的快速乘法利用恒等式:

$$ab = (2^{2n} + 2^n)A_1B_1 + 2^n(A_1 - A_0)(B_0 - B_1) + (2^n + 1)A_0B_0.$$

这样只需三个 n 位乘法，加上一些加减和移位操作。若 $f(n)$ 表示两个 n 位整数相乘所需操作数，则可写成

$$f(2n) = 3f(n) + Cn.$$

等价地，

$$T(N) = 3T(N/2) + O(N),$$

从而

$$T(N) = O(N^{\log_2 3}).$$

2.26 分治递推定理一

设 f 是递增函数，并满足

$$f(n) = af(n/b) + c$$

其中 $a \geq 1$，$b > 1$，$c > 0$。则

$$f(n) = \begin{cases} O(n^{\log_b a}), & a > 1, \\ O(\log n), & a = 1. \end{cases}$$

当 $n = b^k$ 且 $a \neq 1$ 时，还有精确形式：

$$f(n) = C_1 n^{\log_b a} + C_2,$$

其中

$$C_1 = f(1) + \frac{c}{a-1}, \qquad C_2 = -\frac{c}{a-1}.$$

2.27 定理一的展开证明思路

设 $n = b^k$，反复展开：

$$\begin{aligned} f(n) &= af(n/b) + c \\ &= a^2 f(n/b^2) + ac + c \\ &= \cdots \\ &= a^k f(1) + c\sum_{j=0}^{k-1} a^j. \end{aligned}$$

若 $a = 1$，则

$$f(n) = f(1) + ck = f(1) + c\log_b n = O(\log n).$$

若 $a > 1$，则

$$f(n) = a^k f(1) + c\frac{a^k - 1}{a-1}.$$

又因为 $a^k = a^{\log_b n} = n^{\log_b a}$，所以

$$f(n) = C_1 n^{\log_b a} + C_2 = O(n^{\log_b a}).$$

2.28 例子

若

$$f(n) = 5f(n/2) + 3, \qquad f(1) = 7,$$

并令 $n = 2^k$，则 $a = 5, b = 2, c = 3$。由定理一：

$$f(2^k) = 5^k\left(7 + \frac{3}{4}\right) - \frac{3}{4} = \frac{31}{4}5^k - \frac{3}{4}.$$

若 f 是递增函数，则

$$f(n) = O(n^{\log_2 5}).$$

2.29 Master Theorem

若递增函数 f 满足

$$f(n) = af(n/b) + cn^d$$

其中 $n = b^k$，$a \geq 1$，$b > 1$，$c > 0$，$d \geq 0$，则：

$$f(n) = \begin{cases} O(n^d), & a < b^d, \\ O(n^d \log n), & a = b^d, \\ O(n^{\log_b a}), & a > b^d. \end{cases}$$

Master Theorem 判断法

比较“递归子问题总规模增长”a 与“合并代价指数”b^d：若 $a < b^d$，合并层主导；若相等，每层同量级，乘一个 $\log n$；若 $a > b^d$，递归叶子主导。

2.30 归并排序算法与复杂度

Listing 4: 递归归并排序框架

```
Procedure mergesort(L = a1, a2, ..., an):
    if n > 1:
        m = floor(n / 2)
        L1 = a1, a2, ..., am
        L2 = a(m+1), a(m+2), ..., an
        L = merge(mergesort(L1), mergesort(L2))
    return L
```

合并两个有序表时，每一步取两个表首元素中较小者追加到结果表；若某个表为空，则把另一个表剩余元素全部追加。合并代价为 $O(n)$。

因此归并排序满足：

$$M(n) = 2M(n/2) + n.$$

这里 $a = 2, b = 2, d = 1$，有 $a = b^d$，所以

$$M(n) = O(n \log n).$$

2.31 最近点对问题

PPT 提到 closest-pair problem 的分治算法复杂度为

$$O(n \log n).$$

基本思想是：先按横坐标分成左右两半，递归求出左右半区最近距离，再只检查分割线附近宽度为 2δ 的窄带区域。若在递归过程中保持按纵坐标排序，可以将合并阶段控制在线性时间。

3　生成函数

3.1　定义

给定实数序列

$$a_0, a_1, a_2, \ldots, a_k, \ldots$$

它的**普通生成函数 (ordinary generating function)** 是形式幂级数

$$G(x) = a_0 + a_1 x + a_2 x^2 + \cdots + a_k x^k + \cdots = \sum_{k=0}^{\infty} a_k x^k.$$

形式幂级数

在本章中，生成函数主要作为形式幂级数使用，通常不关心收敛性。我们关注的是 x^k 的系数，因为它编码了序列的第 k 项。

3.2　基本例子

序列	生成函数
$3, 3, 3, \ldots$	$\displaystyle\sum_{k=0}^{\infty} 3x^k = \frac{3}{1-x}$
$1, 2, 3, \ldots, k+1, \ldots$	$\displaystyle\sum_{k=0}^{\infty} (k+1)x^k = \frac{1}{(1-x)^2}$
$1, 2, 4, \ldots, 2^k, \ldots$	$\displaystyle\sum_{k=0}^{\infty} 2^k x^k = \frac{1}{1-2x}$
$1, 1, 1, 1, 1$	$1 + x + x^2 + x^3 + x^4$
$\binom{m}{0}, \binom{m}{1}, \ldots, \binom{m}{m}$	$(1+x)^m$

3.3　形式幂级数的运算

设

$$f(x) = \sum_{k=0}^{\infty} a_k x^k, \qquad g(x) = \sum_{k=0}^{\infty} b_k x^k.$$

则:

$$f(x) + g(x) = \sum_{k=0}^{\infty} (a_k + b_k) x^k,$$

$$f(x)g(x) = \sum_{k=0}^{\infty} \left(\sum_{j=0}^{k} a_j b_{k-j} \right) x^k,$$

$$\alpha f(x) = \sum_{k=0}^{\infty} \alpha a_k x^k,$$

$$xf'(x) = \sum_{k=0}^{\infty} k a_k x^k,$$

$$f(\alpha x) = \sum_{k=0}^{\infty} \alpha^k a_k x^k.$$

乘法的意义

两个生成函数相乘时，x^k 的系数来自所有 $j + (k-j) = k$ 的拆分。这正是“总量为 k”的组合计数思想。

3.4 常用展开

$$\frac{1}{1-x} = 1 + x + x^2 + \cdots$$

$$\frac{1}{(1-x)^2} = \sum_{k=0}^{\infty} (k+1) x^k$$

$$\frac{1}{1-ax} = \sum_{k=0}^{\infty} a^k x^k$$

$$\frac{1}{1-4x^2} = 1 + 4x^2 + 16x^4 + 64x^6 + \cdots$$

最后一个式子说明奇数次项系数为 0，偶数次项 x^{2k} 的系数为 4^k。

3.5 广义二项式系数

对实数 u 和非负整数 k，定义

$$\binom{u}{k} = \begin{cases} \dfrac{u(u-1)\cdots(u-k+1)}{k!}, & k > 0, \\ 1, & k = 0. \end{cases}$$

广义二项式定理为：

$$(1+x)^u = \sum_{k=0}^{\infty} \binom{u}{k} x^k.$$

当 $u = -n$ 为负整数时：

$$\binom{-n}{r} = (-1)^r \binom{n+r-1}{r}.$$

因此：

$$(1-x)^{-n} = \sum_{k=0}^{\infty} \binom{n+k-1}{k} x^k.$$

3.6 组合计数中的生成函数

3.6.1 无重复组合

从 n 个不同元素中选 k 个，每个元素只有“不选”和“选一次”两种状态，因此每个元素贡献因子 $(1+x)$。总生成函数为：

$$(1+x)^n = \sum_{k=0}^{n} \binom{n}{k} x^k.$$

所以 x^k 的系数就是 $\binom{n}{k}$。

3.6.2 允许重复组合

从 n 种元素中选 r 个，允许重复。每种元素可以选 $0, 1, 2, \ldots$ 个，因此贡献：

$$1 + x + x^2 + \cdots = \frac{1}{1-x}.$$

总生成函数为：

$$G(x) = \left(1 + x + x^2 + \cdots\right)^n = \frac{1}{(1-x)^n}.$$

由广义二项式展开：

$$\frac{1}{(1-x)^n} = \sum_{r=0}^{\infty} \binom{n+r-1}{r} x^r.$$

所以允许重复的 r 组合数为

$$\binom{n+r-1}{r}.$$

3.6.3 每类至少选一个

若从 n 种对象中选 r 个，并且每种至少选一个，则每种对象贡献

$$x + x^2 + x^3 + \cdots = \frac{x}{1-x}.$$

总生成函数为

$$\left(\frac{x}{1-x}\right)^n = \frac{x^n}{(1-x)^n}.$$

当 $r \geq n$ 时，x^r 的系数为

$$\binom{r-1}{n-1}.$$

3.7 整数解例子

3.7.1 有上下界限制

求

$$e_1 + e_2 + e_3 = 17$$

的非负整数解数，其中

$$2 \le e_1 \le 5, \qquad 3 \le e_2 \le 6, \qquad 4 \le e_3 \le 7.$$

对应生成函数为：

$$(x^2 + x^3 + x^4 + x^5)(x^3 + x^4 + x^5 + x^6)(x^4 + x^5 + x^6 + x^7).$$

答案是该乘积中 x^{17} 的系数，PPT 给出的结果为 3。

3.7.2 奇数非负解

求

$$e_1 + e_2 + e_3 = 17$$

的非负奇数解数。每个变量可取

$$1, 3, 5, \dots$$

所以生成函数为：

$$(x + x^3 + x^5 + \cdots)^3 = \left(\frac{x}{1-x^2}\right)^3 = \frac{x^3}{(1-x^2)^3}.$$

要取 x^{17} 的系数，即取

$$\frac{1}{(1-x^2)^3}$$

中 x^{14} 的系数。令 $y = x^2$，即取 $(1-y)^{-3}$ 中 y^7 的系数：

$$\binom{3+7-1}{7} = \binom{9}{2} = 36.$$

3.8 自动售货机硬币例子

若用价值为 \$1, \$2, \$5 的代币支付 r 美元：

- 若代币投入顺序不重要，则只需数解 $x_1 + 2x_2 + 5x_5 = r$：

  $$G(x) = \frac{1}{(1-x)(1-x^2)(1-x^5)}.$$

  答案是 x^r 的系数。

- 若代币投入顺序重要，则每次投入相当于选择一个权重为 $1, 2, 5$ 的步骤。所有长度不限的序列生成函数为：

  $$1 + (x + x^2 + x^5) + (x + x^2 + x^5)^2 + \cdots = \frac{1}{1 - x - x^2 - x^5}.$$

  答案仍是 x^r 的系数。

3.9 指数生成函数与排列

对于序列 $\{a_n\}$，其**指数生成函数 (exponential generating function, EGF)** 为

$$\sum_{n=0}^{\infty} a_n \frac{x^n}{n!}.$$

指数生成函数适合处理排列问题，因为长度为 n 的排列天然带有 $n!$ 因子。

3.9.1 CHANCE AND CHOICE

短语 `CHANCE AND CHOICE` 中字母多重集为：

$$\{4C, 2H, 2N, 2A, 2E, D, O, I\}.$$

求能形成多少个长度为 4 的不同字符串。对应 EGF 为：

$$G(x) = \left(1 + x + \frac{x^2}{2!} + \frac{x^3}{3!} + \frac{x^4}{4!}\right)\left(1 + x + \frac{x^2}{2!}\right)^4 (1+x)^3.$$

所求数量为 $G(x)$ 中 $x^4/4!$ 的系数，PPT 给出的结果为：

$$3029.$$

3.9.2 偶数个 3 和偶数个 7

求由数字 $1, 3, 5, 7, 9$ 构成的长度为 r 的字符串数量，要求其中 3 和 7 的出现次数均为偶数。数字 3 和 7 各自贡献偶数项：

$$1 + \frac{x^2}{2!} + \frac{x^4}{4!} + \cdots = \frac{e^x + e^{-x}}{2}.$$

数字 $1, 5, 9$ 不受限制，各贡献 e^x。因此

$$G(x) = \left(\frac{e^x + e^{-x}}{2}\right)^2 (e^x)^3 = \frac{1}{4}(e^{5x} + 2e^{3x} + e^x).$$

所以长度为 r 的字符串数为

$$\frac{1}{4}\left(5^r + 2 \cdot 3^r + 1\right).$$

3.10 部分分式分解与系数读取

在用生成函数求递推通项时，经常会得到形如

$$G(x) = \frac{P(x)}{Q(x)}$$

的有理函数。为了从中读出 x^k 的系数，通常需要先把它拆成若干个容易展开的基本形式，例如：

$$\frac{1}{1-ax}, \qquad \frac{1}{(1-ax)^2}, \qquad \frac{1}{1+ax}.$$

这些基本形式可以直接展开为：

$$\frac{1}{1-ax} = \sum_{k=0}^{\infty} a^k x^k,$$

$$\frac{1}{(1-ax)^2} = \sum_{k=0}^{\infty} (k+1) a^k x^k,$$

$$\frac{1}{1+ax} = \sum_{k=0}^{\infty} (-a)^k x^k.$$

分母结构决定拆分形式

若分母中有一个一次因子 $(1-ax)$，则部分分式中写一项

$$\frac{A}{1-ax}.$$

若分母中有重因子 $(1-ax)^2$，则必须写成

$$\frac{A}{1-ax} + \frac{B}{(1-ax)^2}.$$

更一般地，若有 $(1-ax)^m$，则需要从一次到 m 次全部写出：

$$\frac{A_1}{1-ax} + \frac{A_2}{(1-ax)^2} + \cdots + \frac{A_m}{(1-ax)^m}.$$

3.10.1 简单根代，重根导

设

$$G(x) = \frac{3x+5}{(1-x)^2(1+2x)}.$$

因为分母为 $(1-x)^2(1+2x)$，所以应设

$$G(x) = \frac{A}{1-x} + \frac{B}{(1-x)^2} + \frac{C}{1+2x}.$$

对于简单因子 $1+2x$，可以直接用盖住法。令 $1+2x=0$，即 $x=-\frac{1}{2}$，得到

$$C = \left.\frac{3x+5}{(1-x)^2}\right|_{x=-1/2} = \frac{14}{9}.$$

对于重因子 $(1-x)^2$，先求最高次项系数 B：

$$B = \left.(1-x)^2 G(x)\right|_{x=1} = \left.\frac{3x+5}{1+2x}\right|_{x=1} = \frac{8}{3}.$$

再求低一次项系数 A。令

$$H(x) = (1-x)^2 G(x) = \frac{3x+5}{1+2x}.$$

因为这里的因子是 $1-x$，所以

$$A = -H'(1).$$

而

$$H'(x)=\frac{3(1+2x)-2(3x+5)}{(1+2x)^2}=\frac{-7}{(1+2x)^2},$$

所以

$$A=-H'(1)=\frac{7}{9}.$$

因此

$$\frac{3x+5}{(1-x)^2(1+2x)}=\frac{7}{9(1-x)}+\frac{8}{3(1-x)^2}+\frac{14}{9(1+2x)}.$$

展开为幂级数：

$$G(x)=\sum_{k=0}^{\infty}\left(\frac{7}{9}+\frac{8}{3}(k+1)+\frac{14}{9}(-2)^k\right)x^k.$$

所以若 $G(x)=\sum_{k=0}^{\infty}a_kx^k$，则

$$a_k=\frac{7}{9}+\frac{8}{3}(k+1)+\frac{14}{9}(-2)^k.$$

口诀

部分分式分解时，可以记住：简单根直接代，重根先求最高次项，再用求导求低一次项。若因子是 $1-ax$，求导时要注意链式法则带来的系数修正。

3.11 用生成函数解递推

3.11.1 一阶齐次递推

求解

$$a_k=3a_{k-1},\qquad a_0=2.$$

设

$$G(x)=\sum_{k=0}^{\infty}a_kx^k.$$

则

$$xG(x)=\sum_{k=1}^{\infty}a_{k-1}x^k.$$

利用递推关系：

$$G(x)-3xG(x)=a_0+\sum_{k=1}^{\infty}(a_k-3a_{k-1})x^k$$

$$=2.$$

所以

$$G(x)=\frac{2}{1-3x}=2\sum_{k=0}^{\infty}3^kx^k.$$

因此

$$a_k=2\cdot 3^k.$$

3.11.2 偶数个 0 的码字

PPT 中的例子：有效码字是含偶数个 0 的十进制 n 位串，且

$$a_n = 8a_{n-1} + 10^{n-1}, \qquad a_1 = 9.$$

取 $a_0 = 1$ 后，设

$$G(x) = \sum_{n=0}^{\infty} a_n x^n.$$

对 $n \geq 1$ 求和：

$$G(x) - 1 = 8xG(x) + \frac{x}{1-10x}.$$

解得：

$$G(x) = \frac{1-9x}{(1-8x)(1-10x)} = \frac{1}{2}\left(\frac{1}{1-8x} + \frac{1}{1-10x}\right).$$

因此

$$a_n = \frac{1}{2}(8^n + 10^n).$$

3.12 用生成函数证明恒等式

证明：

$$\sum_{k=0}^{n} \binom{n}{k}^2 = \binom{2n}{n}.$$

由二项式定理，$\binom{2n}{n}$ 是 $(1+x)^{2n}$ 中 x^n 的系数。另一方面：

$$(1+x)^{2n} = \left((1+x)^n\right)^2.$$

而

$$(1+x)^n = \sum_{k=0}^{n} \binom{n}{k} x^k.$$

平方后，x^n 的系数为

$$\binom{n}{0}\binom{n}{n} + \binom{n}{1}\binom{n}{n-1} + \cdots + \binom{n}{n}\binom{n}{0}.$$

由于

$$\binom{n}{n-k} = \binom{n}{k},$$

该系数等于

$$\sum_{k=0}^{n} \binom{n}{k}^2.$$

两者都是 $(1+x)^{2n}$ 中 x^n 的系数，因此相等。

4　容斥原理及其应用

4.1　基本形式

设 $A_1, A_2, \ldots, A_n$ 是有限集合，则：

$$\left|\bigcup_{i=1}^{n} A_i\right| = \sum_{1\le i\le n} |A_i| - \sum_{1\le i<j\le n} |A_i\cap A_j| + \sum_{1\le i<j<k\le n} |A_i\cap A_j\cap A_k| - \cdots + (-1)^{n+1}|A_1\cap A_2\cap\cdots\cap A_n|.$$

为什么要加减交替

若某个元素同时属于 r 个集合，它在单集合求和中被数了 $\binom{r}{1}$ 次，在两两交集中被减了 $\binom{r}{2}$ 次，在三重交集中被加了 $\binom{r}{3}$ 次。总贡献为

$$\binom{r}{1} - \binom{r}{2} + \binom{r}{3} - \cdots + (-1)^{r+1}\binom{r}{r} = 1.$$

所以每个属于并集的元素最终恰好被数一次。

4.2　三集合形式

对三个集合 A, B, C：

$$|A\cup B\cup C| = |A| + |B| + |C| - |A\cap B| - |A\cap C| - |B\cap C| + |A\cap B\cap C|.$$

4.3　语言课程例子

有 1232 名学生学过西班牙语，879 名学过法语，114 名学过俄语；其中 103 名学过西班牙语和法语，23 名学过西班牙语和俄语，14 名学过法语和俄语。若共有 2092 名学生至少学过其中一门语言，求三门都学过的学生数。

设 S, F, R 分别表示学过西班牙语、法语、俄语的学生集合。由容斥：

$$|S\cup F\cup R| = |S| + |F| + |R| - |S\cap F| - |S\cap R| - |F\cap R| + |S\cap F\cap R|.$$

代入：

$$2092 = 1232 + 879 + 114 - 103 - 23 - 14 + |S\cap F\cap R|.$$

解得：

$$|S\cap F\cap R| = 7.$$

4.4 容斥原理的应用

4.5 性质形式

设全集共有 N 个元素，P_i 是第 i 个性质。记 $N(P_{i_1}P_{i_2}\cdots P_{i_k})$ 为同时具有这些性质的元素数。则没有任何性质 $P_1,\ldots,P_n$ 的元素数为：

$$N(P_1'P_2'\cdots P_n') = N - \sum_i N(P_i) + \sum_{i<j} N(P_iP_j) - \sum_{i<j<k} N(P_iP_jP_k) + \cdots + (-1)^n N(P_1P_2\cdots P_n).$$

4.6 有上界的非负整数解

求非负整数解个数：

$$x_1 + x_2 + x_3 = 11,$$

其中

$$x_1 \le 3, \qquad x_2 \le 4, \qquad x_3 \le 6.$$

先不考虑上界，总数为

$$\binom{11+3-1}{3-1} = \binom{13}{2} = 78.$$

令性质

$$P_1 : x_1 > 3, \qquad P_2 : x_2 > 4, \qquad P_3 : x_3 > 6.$$

则：

$$N(P_1) = \binom{7+2}{2} = 36, \quad N(P_2) = \binom{6+2}{2} = 28, \quad N(P_3) = \binom{4+2}{2} = 15.$$

两两交集：

$$N(P_1P_2) = \binom{2+2}{2} = 6, \qquad N(P_1P_3) = \binom{0+2}{2} = 1, \qquad N(P_2P_3) = 0.$$

三重交集为 0。所以合法解数为

$$78 - 36 - 28 - 15 + 6 + 1 = 6.$$

4.7 Eratosthenes 筛法

容斥可用于计算不超过某个正整数的素数数量。关键事实是：若一个整数是合数，则它一定有一个不超过其平方根的素因子。

例如求不超过 100 的素数。因为

$$\sqrt{100} = 10,$$

只需考虑素数 $2,3,5,7$ 的倍数。先保留 $2,3,5,7$，再在 $\{2,3,\ldots,100\}$ 中排除所有能被 $2,3,5,7$ 整除的数，剩下的就是大于 10 的素数。该过程就是 Eratosthenes 筛法的思想。

4.8 满射数量

从一个 m 元集合到一个 n 元集合的函数共有 n^m 个。若要求是**满射 (onto function)**，则值域中每个元素都至少被映到一次。

PPT 例子：从 6 元集合到 3 元集合的满射有多少个？设陪域元素为 b_1, b_2, b_3，性质 P_i 表示 b_i 不在值域中。则：

$$N = 3^6.$$

若某个 b_i 不在值域中，函数只能映到剩下两个元素：

$$N(P_i) = 2^6.$$

若两个指定元素不在值域中，只能映到剩下一个元素：

$$N(P_iP_j) = 1^6 = 1.$$

三个都不在值域中不可能：

$$N(P_1P_2P_3) = 0.$$

所以满射数量为

$$3^6 - 3 \cdot 2^6 + 3 \cdot 1^6 = 729 - 192 + 3 = 540.$$

一般地，若 $m \geq n$，从 m 元集合到 n 元集合的满射数量为

$$\sum_{j=0}^{n} (-1)^j \binom{n}{j} (n-j)^m.$$

它也等于

$$n!\, S(m, n),$$

其中 $S(m, n)$ 是第二类 Stirling 数。

4.9 错排

错排 (derangement) 是没有任何对象留在原位置的排列。例如 21453 是 12345 的错排，因为每个位置上的数字都变了；而 21543 不是错排，因为 4 仍在原位置。

设 D_n 为 n 个对象的错排数。令性质 P_i 表示第 i 个对象留在原位置。由容斥可得：

$$D_n = n! \left(1 - \frac{1}{1!} + \frac{1}{2!} - \frac{1}{3!} + \cdots + (-1)^n \frac{1}{n!}\right).$$

4.10 帽子寄存问题

有 n 位顾客寄存帽子，工作人员忘记贴号码，归还时随机分配帽子。没有任何人拿回自己帽子的概率为：

$$\frac{D_n}{n!} = 1 - \frac{1}{1!} + \frac{1}{2!} - \frac{1}{3!} + \cdots + (-1)^n \frac{1}{n!}.$$

当 n 趋于无穷大时，该概率趋近于

$$\frac{1}{e}.$$

5 本章总结 (Summary)

- 递推关系用前面若干项定义当前项，必须配合初始条件才能唯一确定序列。
- 建立递推时常用分类讨论：最后一步是什么、是否选择某对象、某对象是否在最优解中。
- 动态规划把递推关系变成可执行算法，核心是状态、转移、初始化和答案。
- 线性齐次常系数递推可用特征方程求解；不同根给 r^n，重根给 $n^j r^n$。
- 线性非齐次递推的通解等于关联齐次通解加一个特解。
- 分治递推通常形如 $f(n) = af(n/b) + g(n)$，Master Theorem 可快速估计复杂度。
- 普通生成函数把序列编码为幂级数系数，适合组合计数和递推求解。
- 指数生成函数适合处理带排列因素的问题。
- 容斥原理通过加减交替修正重复计数，适合处理“至少一个限制被违反”或“没有任何性质”的计数问题。
- 满射、错排、Eratosthenes 筛法和有上界整数解都是容斥原理的典型应用。

5.1 PPT 作业汇总

- 8.1：P.536：10，24，25，53，56。
- 8.2：P.550：20，30，40，35，52。
- 8.3：P.561：12，13，20，24，29–31，其中 29–31 与 Theorem 2 证明相关。
- 8.4：P.575：11，17，25，27，35，38，44。
- 8.5：P.584：7，13，22。
- 8.6：P.591：7，9，11，13，17。

5.2 术语对照表

中文术语	英文术语
递推关系	Recurrence Relation
初始条件	Initial Conditions
递推关系的阶	Degree / Order of a Recurrence Relation
解	Solution
动态规划	Dynamic Programming
记忆化	Memoization
最长递增子序列	Longest Increasing Subsequence, LIS
最长公共子序列	Longest Common Subsequence, LCS
兼容讲座 / 区间	Compatible Talks / Intervals
线性齐次常系数递推	Linear Homogeneous Recurrence Relation with Constant Coefficients
线性非齐次常系数递推	Linear Nonhomogeneous Recurrence Relation with Constant Coefficients
特征方程	Characteristic Equation
特征根	Characteristic Root
特解	Particular Solution
关联齐次递推	Associated Homogeneous Recurrence Relation
分治算法	Divide-and-Conquer Algorithm
Master Theorem	主定理
归并排序	Merge Sort
快速整数乘法	Fast Multiplication of Integers
生成函数	Generating Function
普通生成函数	Ordinary Generating Function
指数生成函数	Exponential Generating Function
形式幂级数	Formal Power Series
广义二项式系数	Extended Binomial Coefficient
容斥原理	Principle of Inclusion-Exclusion
Eratosthenes 筛法	Sieve of Eratosthenes
满射	Onto Function / Surjection
错排	Derangement
帽子寄存问题	Hatcheck Problem